

Graph RAG

👤 Last edited by	Ⓐ Abdul Moiz
🕒 Last edited time	May 15, 2026 8:06 PM
⚙️ Status	Not started

Basic Overview:

Standard RAG uses embeddings where text is converted into vectors inside a latent/vector space. Retrieval happens based on similarity search, but high similarity does not always mean the retrieved information is actually correct or relevant. It uses semantic nearby concept lets say e have a python document it has evrything regarding python and we ask who created python it may retrieve other things too this is also vector less rag

Graph rag:

how it works is that we construct the knowledge graph from our corpus (dataset,document,large chunk of data) graph such that $G = (V,E)$, where v = nodes and e = entities. how it works:

The Indexing Phase (Data to Graph)

- Entity & Relationship Extraction:** Documents are chunked, and an LLM is prompted to extract structured data—identifying nodes (e.g., "FastAPI", "Backend") and edges with descriptive weights (e.g., "FastAPI" → `USED_FOR` → "Backend" [description: "handles asynchronous routing"]).
- Graph Construction & Resolution:** The raw extractions are deduplicated (Entity Resolution). "Fast API" and "FastAPI" are merged into a single node.
- Community Detection:** We apply graph clustering algorithms—specifically the **Leiden algorithm**. This partitions the graph into hierarchical communities (clusters of densely connected nodes).
- Community Summarization:** *This is the secret sauce.* An LLM generates a summary for *every single community* at various levels of the hierarchy. You

now have high-level summaries of the entire corpus and low-level summaries of specific niches.

The Query Phase (Routing the Request)

When a user asks a question, GraphRAG routes it to one of two pipelines:

- **Local Search (Entity-Centric):** Best for specific queries. We use standard vector search to find relevant nodes in the graph, then retrieve their connected edges and neighboring nodes. This sub-graph is injected into the LLM context window.
- **Global Search (Corpus-Centric):** Best for questions like "What are the overarching themes in these notes?" We run a map-reduce operation over the pre-generated *community summaries*, allowing the LLM to answer questions about the entire dataset without needing a million-token context window.

Some implementation Hacks:

- it takes time for knowledge graph generation so we have to preferably use a message broker so it can perform the indexing phase asynchronously main principle we used here is Named Entity Recognition
- storing the data?

Graph Database (Neo4j): Stores the actual nodes, edges, and community structures. Cypher is the standard query language here.

Vector Database (Milvus or Qdrant): You still need vector search! You embed the *descriptions* of the nodes and the *community summaries*.

The Bridge: Every node in Neo4j contains a metadata field with its corresponding Vector DB ID.

summarization in simple words:

Standard RAG (Dense Vector Search) is like an index at the back of a textbook. It points you to specific pages. GraphRAG is like having the author's brain—it

understands how the chapters connect.

Issue: GraphRAG introduces massive indexing overhead. You are paying LLMs to read every single document just to extract the graph structure before a user even asks a question. If standard RAG gets you 90% accuracy on a fast, cheap vector search, don't overengineer it.

Leiden Community Detection Algorithm:

the real thing is we can have 10000+ nodes and edgess and we still cant send entirely to the llm so what we do is we use leiden community detection algo

Leiden community detection is a graph clustering algorithm that groups related nodes into tightly connected "communities".

Core idea:

- Nodes that are strongly connected end up in the same group.
- Weakly connected nodes are separated into different groups.

How it works (simple view):

1. Start with each node as its own group.
2. Move nodes into neighboring groups if it improves connectivity.
3. Repeat until groups stabilize.
4. Compress each group into a single "super-node" and repeat on the reduced graph.

Result:

- A large graph is reduced into meaningful clusters (topics).

In GraphRAG:

- Each community becomes a topic (e.g., "FastAPI + ASGI", "Redis caching").
- These clusters are later summarized and used for global search instead of scanning the entire dataset.

but how do we know that our grouping is good or not? so we have modularity it tells how good our grouping is

The Practical Implementation (Map-Reduce):

Once Leiden groups your graph into these hierarchical communities, you do something brilliant: **You pre-generate summaries.**

- You pass all the nodes and edges of a *Level 1* community to an LLM and ask: *"Summarize what this community represents."*
- You do this for every community. Then, you use those summaries to generate even higher-level summaries for Level 2.

The Query Phase: When that global API request hits your FastAPI backend, you don't search the raw graph. You run a Map-Reduce operation. You map the user's query against your pre-generated *Community Summaries*, filter out the irrelevant ones, and reduce the relevant summaries into a final, highly accurate global answer.